Doubly Linked List (Basic Operations)

(Insertion at the end)

```
void insert()
{
  // Rest of the code

Node* newnode = new Node(val); // creating new node

if(head == NULL)
{
  head = tail = newnode;
}
// if 1st node, point both head & tail to newnode

else
{
  newnode->prev = tail;
  tail->next = newnode;
  tail = newnode;
}
// if not first then put tail into the prev of newnode.
// then put newnode in the next of tail that is now 2nd last node.
// Now point tail to newnode
```

(Insertion at beginning)

```
void insertatb()
{
   ... // ROC
   Node* newnode = new Node(val); // creating new
   if(head==NULL)
   {
     head=tail=newnode;
   }
   //if 1st node, then point head & tail to newn
   else
   {
     newnode->next=head;
     head->prev=newnode;
     head=newnode;
   }
   //if not 1st then put head in the next
     of newnode as it is inserted at begg
   // Now put newnode in the prev of head
   // Lastly point head to this newnode.
```

(Insert after value)

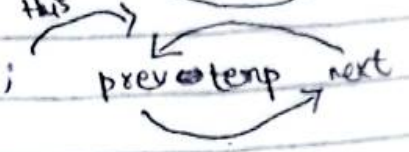
```
void insertaftval()
{
   if(head==NULL){return;}
   // if no node; so return
```

```
Node* temp = head;   // to find value

.... // RDC
while(temp!=NULL && temp->data!=target)
{
  //Run loop until temp becomes NULL or
    we found target value.
    temp = temp->next;
    //traversing temp /incrementing

}

if(temp==NULL){return;}   //if val not found

... // RDC
Node* newnode = new Node(val);  //created node
newnode->next = t->next;   //put next of temp into
                             next of this newnode
newnode->prev = temp;    //put temp into prev
                           of newnode

if(temp->next ~~express~~ !=NULL)     //if next of t!=NULL
{
  temp->next->prev = newnode;  // put newnode
                                 into the prev
}                                of next of temp
                               // Dry run clearly
temp->next = newnode;            explains.
//put newnode into next of temp

if(temp==tail)        // if temp is at end
{                          ~~tmp~~
  tail = newnode;     // update tail
}
```

(Void Delete by Value)

```
void dbl()
{
  if(head == NULL)      // if no node
  { return; }

  ... // ROC

  if (head -> data == val)      // if data is at head
  {
    Node* temp = head;          // store head in temp
    head = head -> next;        // move head to it's next

    if(head != NULL)            // if head ≠ NULL
    {
      head -> prev = NULL;      // mov NULL into prev of
    }                           this new head
    else                        // if head = NULL
    {
      tail = NULL;              // point tail -> NULL als
    }
    delete temp;                // delete the node of
    return
  }

Node* temp = head;              // for traversing

while(temp != NULL && temp -> data != val)
{
  temp = temp -> next;
}     // same as before
```

```
if (temp == NULL){ return ;}      // if value not found

if (temp->prev != NULL )          // if temp->prev ≠ NULL
{
  temp->prev->next = temp->next;  // mov next of temp
}                                    int next of prev of
                                     temp.
if(temp->next != NULL)            // prev temp next
{
  temp->next->prev = temp->prev;
}
```




```
  if(temp == tail)      // if temp is at end
  {
    tail = temp->prev;  // mov tail to its previous
  }
  delete temp;          // delet that node of val
}
```

(search by value)

```
bool sbv()
{
  if(head == NULL) { return ; }

  ...
  Node* temp = head;
  ~~Node* newNode = new Node;~~

  while(temp != NULL)          // same as SSL
  { if (temp->data == val)
    { return true;
    } temp = temp->next;
  }
}
```

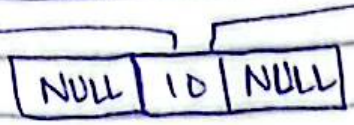
(DRY_RUN)

• Insertionatend():

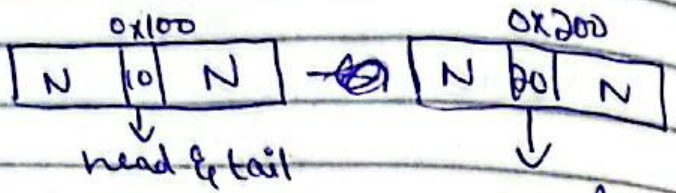
⇒ First Node:

before → head = tail = NULL;

⇒ After newnode:

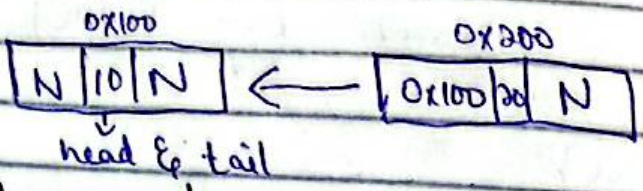


//Now if 2nd node or more nodes insertion;



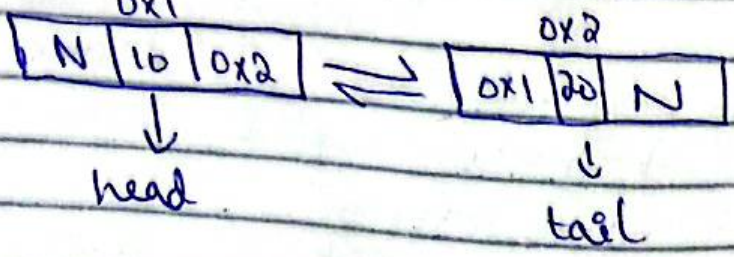
⇒ Now:

newnode → prev = tail;



tail → next = newnode;

tail = newnode; (update tail)



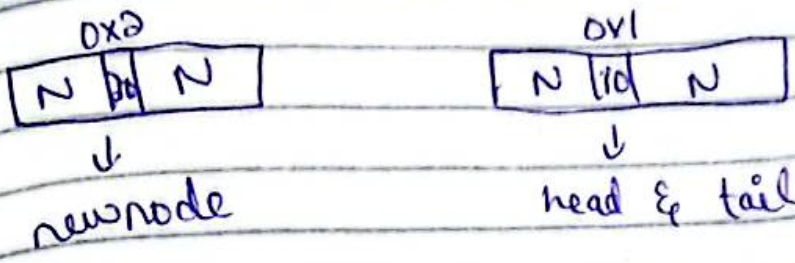
and So on

1) Insertionatbeginning():

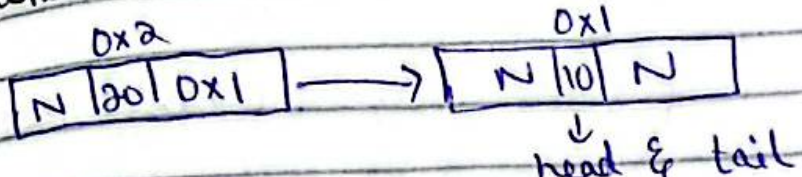
=> First Node:

(Same)

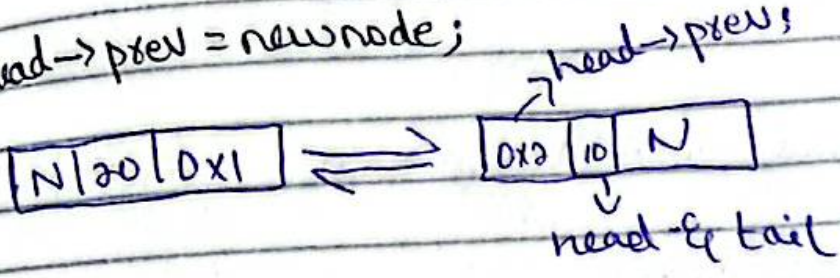
=> Second or more nodes:



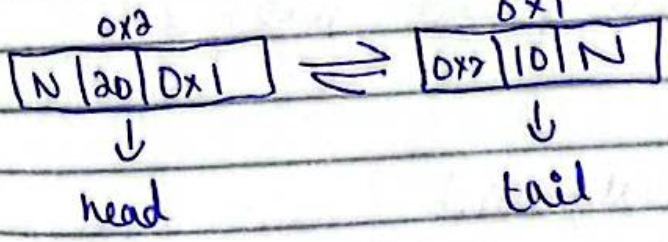
newnode->next = head;



head->prev = newnode;



head = newnode; (update head)



& so on

Insertaftervalue():

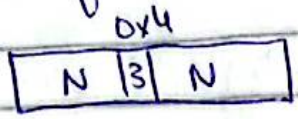
Node* temp=head; For traverse

while(→){ } // to find value after which we have to insert

Suppose:

0x1	N	1	0x2
0x2	0x1	2	0x3
0x3	0x2	3	N

→ temp is here, we will insert after this:

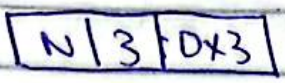


→ newnode created

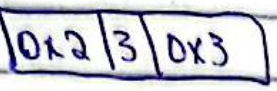
temp->next;

=> now:

newnode->next=temp->next;

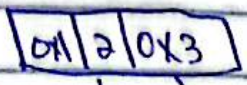


newnode->prev=temp;

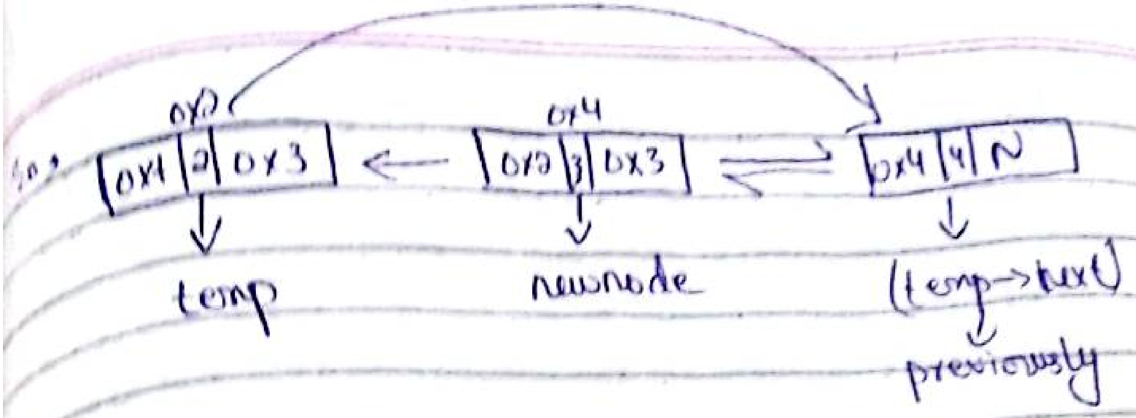


→ Now:

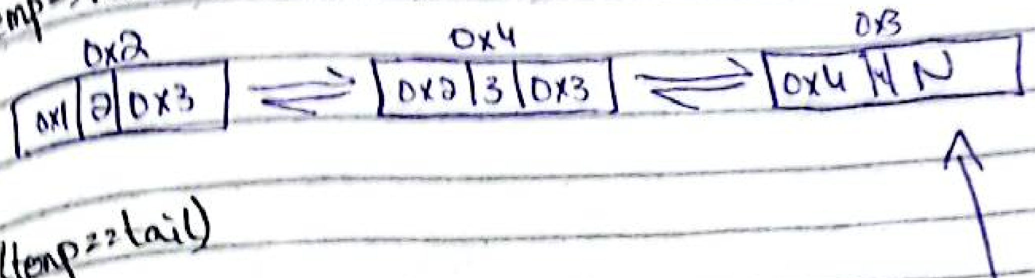
```
if(temp->next!=NULL)
{
    temp->next->prev=newnode;
}
```



temp temp->next temp->next



temp->next = newnode;



if(temp==tail)
{
tail=newnode; // simply update tail if this "was" NULL.
}

:) Delete by value():

// if the head is about to del just shift head to it's next & move NULL into the prev of new head if it is not NULL if NULL then point tail to NULL also.

Node* temp=head; // For traversing

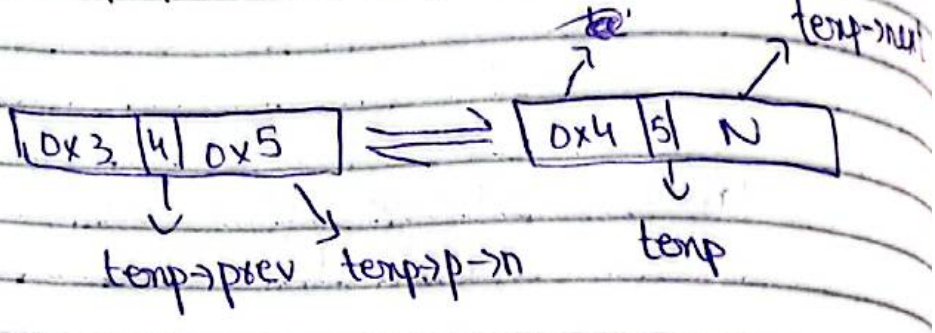
while(—){—} // until we found val to be del, move temp to it's next for every iteration.

if(temp==NULL) // if val not found
{ return; }

0x1	N	1	0x2
0x2	0x1	2	0x3
0x3	0x2	3	0x4
0x4	0x3	4	0x5
0x5	0x4	5	N

→ Suppose temp is here now

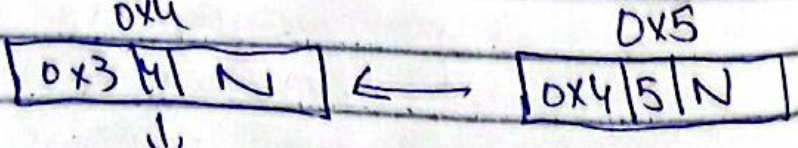
```
if (temp->prev != NULL)
{
    temp->prev->next = temp->next;
}
```



So,


```
if (temp->next != NULL)
{
    ---- }   // as it is NULL

if (temp == tail)
{
    tail = temp->prev;
}
```

So,



delete temp now.

Update search by value():

Node* temp=head

Simple traversal, if val found prompt it & return true otherwise false

Void Update():

Node* temp=head; // for traversal

Suppose:

0x1	N	1	0x2
0x2	0x1	2	0x3
0x3	0x5	3	0x4
0x4	0x3	4	N

Let's update (0x3) (3) to (5)

if temp = 0x1:

temp != NULL ✓
temp->d = val ✗
temp = t->n;

if t = 0x2:

t != NULL ✓
t->d = val ✗
t = t->n

if t = 0x3

t != Null ✓
t->d = val ✓
{
t->d = 5;}

So,

0x1	N	1	0x2
0x2	0x1	2	0x3
0x3	0x2	5	0x4
0x4	0x3	4	N

↓

our new list

Void display():

Same as SLL